

EXP -6 Fibonacci series using Recursion

```
#include <stdio.h>

int fibonacci(int n)
{
    if (n == 0)
        return 0;
    else if (n == 1)
        return 1;
    else
        return fibonacci(n - 1) + fibonacci(n - 2);
}

int main()
{
    int n, i;
    printf("Enter the number of terms: ");
    scanf("%d", &n);
    printf("Fibonacci Series: ");
    for (i = 0; i < n; i++)
    {
        printf("%d ", fibonacci(i));
    }
    return 0;
}
```

Output

Enter the number of terms: 5

Fibonacci Series: 0 1 1 2 3

=== Code Execution Successful ===

EXP – 7 Array operations Insertion

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int arr[100], n, i, pos, element, choice;
```

```
    printf("Enter the number of elements: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter the array elements:\n");
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
    printf("\n1. Insert at Beginning");
```

```
    printf("\n2. Insert at End");
```

```
    printf("\n3. Insert at Any Position");
```

```
    printf("\nEnter your choice: ");
```

```
    scanf("%d", &choice);
```

```
    printf("Enter the element to insert: ");
```

```
    scanf("%d", &element);
```

```
    switch(choice)
```

```
    {
```

```
        case 1: // Insert at Beginning
```

```

        for(i = n; i > 0; i--)
        {
            arr[i] = arr[i - 1];
        }
        arr[0] = element;
        n++;
        break;
case 2: // Insert at End
        arr[n] = element;
        n++;
        break;
case 3: // Insert at Any Position
        printf("Enter the position: ");
        scanf("%d", &pos);
        for(i = n; i >= pos; i--)
        {
            arr[i] = arr[i - 1];
        }
        arr[pos - 1] = element;
        n++;
        break;
default:
        printf("Invalid Choice");
        return 0;
}

printf("\nArray after insertion:\n");
for(i = 0; i < n; i++)
{
    printf("%d ", arr[i]);

```

```

    }
    return 0;
}

```

Output

```

Enter the number of elements: 6
Enter the array elements:
10 25 23 24 65 25

1. Insert at Beginning
2. Insert at End
3. Insert at Any Position
Enter your choice: 1
Enter the element to insert: 52

Array after insertion:
52 10 25 23 24 65 25

=== Code Execution Successful ===

```

EXP – 8 Array operations in deletion

```

#include <stdio.h>

int main()
{
    int arr[100], n, i, pos, choice;
    printf("Enter the number of elements: ");
    scanf("%d", &n);
    printf("Enter the array elements:\n");
    for(i = 0; i < n; i++)
    {

```

```

scanf("%d", &arr[i]);
}
printf("\n1. Delete at Beginning");
printf("\n2. Delete at End");
printf("\n3. Delete at Any Position");
printf("\nEnter your choice: ");
scanf("%d", &choice);
switch(choice)
{
    case 1: // Delete at Beginning
        for(i = 0; i < n - 1; i++)
        {
            arr[i] = arr[i + 1];
        }
        n--;
        break;
    case 2: // Delete at End
        n--;
        break;
    case 3: // Delete at Any Position
        printf("Enter the position to delete: ");
        scanf("%d", &pos);
        for(i = pos - 1; i < n - 1; i++)
        {
            arr[i] = arr[i + 1];
        }

```

```

        n--;
        break;
default:
    printf("Invalid Choice");
    return 0;
}
printf("\nArray after deletion:\n");
for(i = 0; i < n; i++)
{
    printf("%d ", arr[i]);
}
return 0;
}

```

Output

```

Enter the number of elements: 5
Enter the array elements:
10 520 62 55 2

```

```

1. Delete at Beginning
2. Delete at End
3. Delete at Any Position
Enter your choice: 2

```

```

Array after deletion:
10 520 62 55

```

```

=== Code Execution Successful ===

```

EXP -9 Stack operations such as PUSH, POP and Display

```
#include <stdio.h>

#define MAX 5

int stack[MAX];

int top = -1;

void push()
{
    int item;

    if(top == MAX - 1)
    {
        printf("Stack Overflow\n");
        return;
    }

    printf("Enter element to push: ");
    scanf("%d", &item);
    top++;
    stack[top] = item;
    printf("%d pushed into stack\n", item);
}

void pop()
{
    if(top == -1)
    {
        printf("Stack Underflow\n");
        return;
    }

    printf("Deleted element: %d\n", stack[top]);
```

```
        top--;
    }
void display()
{
    int i;
    if(top == -1)
    {
        printf("Stack is Empty\n");
        return;
    }
    printf("Stack elements are:\n");
    for(i = top; i >= 0; i--)
    {
        printf("%d\n", stack[i]);
    }
}
int main()
{
    int choice;
    do
    {
        printf("\n--- STACK OPERATIONS ---\n");
        printf("1. PUSH\n");
        printf("2. POP\n");
        printf("3. DISPLAY\n");
        printf("4. EXIT\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch(choice)
```



```

{
    case 1:
        push();
        break;
    case 2:
        pop();
        break;
    case 3:
        display();
        break;
    case 4:
        printf("Exiting...\n");
        break;
    default:
        printf("Invalid Choice\n");
}
} while(choice != 4);

return 0;

```

Output

```

--- STACK OPERATIONS ---
1. PUSH
2. POP
3. DISPLAY
4. EXIT
Enter your choice: 1
Enter element to push: 56
56 pushed into stack

--- STACK OPERATIONS ---
1. PUSH
2. POP
3. DISPLAY
4. EXIT
Enter your choice: 1
Enter element to push: 56
56 pushed into stack

```

EXP – 10 Insertion of a New Node at the Beginning, End and Any Position of a Singly Linked List

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node
```

```
{
```

```
    int data;
```

```
    struct Node *next;
```

```
};
```

```
struct Node *head = NULL;
```

```
void insertBeginning(int value)
```

```
{
```

```
    struct Node *newNode;
```

```
    newNode = (struct Node *)malloc(sizeof(struct Node));
```

```
    newNode->data = value;
```

```
    newNode->next = head;
```

```
    head = newNode;
```

```
}
```

```
void insertEnd(int value)
```

```
{
```

```
    struct Node *newNode, *temp;
```

```
    newNode = (struct Node *)malloc(sizeof(struct Node));
```

```
newNode->data = value;  
newNode->next = NULL;
```

```
if(head == NULL)  
{  
    head = newNode;  
}
```

```
else
```

```
{  
    temp = head;
```

```
    while(temp->next != NULL)  
    {  
        temp = temp->next;  
    }
```

```
    temp->next = newNode;  
}  
}
```

```
void insertAtPosition(int value, int pos)
```

```
{  
    struct Node *newNode, *temp;  
    int i;
```

```
    newNode = (struct Node *)malloc(sizeof(struct Node));  
    newNode->data = value;
```

```
    if(pos == 1)
```

```
{  
    newNode->next = head;  
    head = newNode;  
    return;  
}
```

```
temp = head;
```

```
for(i = 1; i < pos - 1 && temp != NULL; i++)
```

```
{  
    temp = temp->next;  
}
```

```
if(temp == NULL)
```

```
{  
    printf("Invalid Position\n");  
    return;  
}
```

```
newNode->next = temp->next;
```

```
temp->next = newNode;
```

```
}
```

```
void display()
```

```
{  
    struct Node *temp = head;
```

```
    printf("Linked List: ");
```

```

while(temp != NULL)
{
    printf("%d -> ", temp->data);
    temp = temp->next;
}

printf("NULL\n");
}

int main()
{
    int choice, value, pos;

    do
    {
        printf("\n--- LINKED LIST INSERTION OPERATIONS ---\n");
        printf("1. Insert at Beginning\n");
        printf("2. Insert at End\n");
        printf("3. Insert at Any Position\n");
        printf("4. Display\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
            case 1:
                printf("Enter value: ");
                scanf("%d", &value);

```

```
insertBeginning(value);  
break;
```

case 2:

```
printf("Enter value: ");  
scanf("%d", &value);  
insertEnd(value);  
break;
```

case 3:

```
printf("Enter value: ");  
scanf("%d", &value);  
printf("Enter position: ");  
scanf("%d", &pos);  
insertAtPosition(value, pos);  
break;
```

case 4:

```
display();  
break;
```

case 5:

```
printf("Exiting...\n");  
break;
```

default:

```
printf("Invalid Choice\n");
```

```
}
```

```
} while(choice != 5);
```

```
return 0;
```

```
} 0;
```

```
return 0;
```

Output

```
--- LINKED LIST INSERTION OPERATIONS ---
```

1. Insert at Beginning
2. Insert at End
3. Insert at Any Position
4. Display
5. Exit

```
Enter your choice: 1
```

```
Enter value: 5
```

```
--- LINKED LIST INSERTION OPERATIONS ---
```

1. Insert at Beginning
2. Insert at End
3. Insert at Any Position
4. Display
5. Exit

```
Enter your choice: 2
```

```
Enter value: 7
```

```
}
```